Introduction to Computer Graphics

HW4

Clash Royale: Usagi v.s. Chiikawa

Group 18

112550011 李佑軒
112550153 蔡凱旭
112550183 蔡承志

Introduction

Story: Chiikawa and Usagi are having a match in Clash Royale, a famous smartphone game. When Chiikawa places its favorite card, the infamous Mega Knight, it thinks it's gonna win, and thus spamming the emote trying to provoke Usagi. However, Usagi fires a rocket to Chiikawa's king tower, crushes the tower and also Chiikawa itself. At the end, Usagi also respond with the friendly Goblin crying emote.

We will make an animation of the above story.

Implementation Details

Under the environment of HW3 static version,

Blender using:

- Color: use edit mode, choose the plain to color those originally uncoloured model (crown, blue and red princess)
- Change file type: import different object types into blender, connect the texture, then export as .obj and .mlt for shader using

Load Royal Arena:

import .fbx into blender, we discover that there are three objects, separately export them as .obj and .mlt, bind their texture, and display it by the same way as HW3.

Characters

We load the obj and mtl file of characters(Usagi(the yellow one),Chiikawa(the white one),princess, and mega knight(the black one)), and display it in the arena. Also,we made an idle animation for Chiikawa and Usagi, which is to stretch,squash and bounce up and down shown in the demo video, and we do it by using a sine wave to create smooth up-and-down bouncing and stretching motion

```
// Usagi idle bounce animation (slime-like)
usagiBounceTime += deltaTime * usagiBounceSpeed;
float bounceOffset = sin(usagiBounceTime) * usagiBounceHeight;
// Squash when down, stretch when up
float squashStretch = 1.0f + sin(usagiBounceTime) * usagiSquashAmount;
float inverseSquash = 1.0f - sin(usagiBounceTime) * usagiSquashAmount * 0.5f;

usagiModelMatrix = glm::mat4(1.0f);
usagiModelMatrix = glm::translate(usagiModelMatrix, usagiBasePos + glm::vec3(0.0f, bounceOffset, 0.0f));
usagiModelMatrix = glm::scale(usagiModelMatrix, glm::vec3(20.0f * inverseSquash, 20.0f * squashStretch, 20.0f * inverseSquash));
usagiModelMatrix = glm::rotate(usagiModelMatrix, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
```

We also made a walking animation for mega knight by first locating the leg's position and then using sin wave displacement on each vertex of the legs.

Emotes

Emotes are the laughing and green goblin crying gif shown in the demo video. We use texture mapping to map photos to the object, which is similar to hw3, and since the emote is originally a gif, we extract the frames of the gif to multiple static png files and then display each of them for 30ms to reach a similar effect.

Particle Explosion via Geometry Shader and fragment shader:

The explosion effect is implemented using a Geometry Shader that transforms point primitives into camera-facing quads.

```
#version 330 core
layout (points) in;
layout (triangle_strip, max_vertices = 4) out;
```

The shader takes points as input and outputs a layout(triangle_strip, max_vertices = 4). For every incoming point—representing the particle's center—the shader generates four vertices to form a rectangular quad.

```
float normalizedAge = age / 3.0f;
if (normalizedAge < 0.3f) {
    float t = normalizedAge / 0.3f;
    particleColor.rgb = mix(vec3(1.0f, 1.0f, 0.3f), vec3(1.0f, 0.5f, 0.0f), t);
} else if (normalizedAge < 0.6f) {
    float t = (normalizedAge - 0.3f) / 0.3f;
    particleColor.rgb = mix(vec3(1.0f, 0.5f, 0.0f), vec3(1.0f, 0.1f, 0.0f), t);
} else {
    float t = (normalizedAge - 0.6f) / 0.4f;
    particleColor.rgb = mix(vec3(1.0f, 0.1f, 0.0f), vec3(0.2f, 0.2f, 0.2f), t);
}
```

The visual characteristics of each particle are driven by its age attribute, normalized over a 3-second lifespan. To simulate the thermal cooling of an explosion. Particles transition from high-intensity **Yellow** (age < 0.3) to **Orange**, then to **Red**, and finally to a **Dark Gray/Smoke** color (0.6 < age < 1.0) .

```
vec3 cameraRight = vec3(view[0][0], view[1][0], view[2][0]);
vec3 cameraUp = vec3(view[0][1], view[1][1], view[2][1]);

vec4 viewPos = view * center;

vec4 offsets[4] = vec4[](
    viewPos + vec4(-size * cameraRight - size * cameraUp, 0.0f),
    viewPos + vec4(size * cameraRight - size * cameraUp, 0.0f),
    viewPos + vec4(-size * cameraRight + size * cameraUp, 0.0f),
    viewPos + vec4(size * cameraRight + size * cameraUp, 0.0f)
);
```

To ensure that particles maintain a consistent circular or rectangular appearance regardless of the camera's orientation, a **billboarding** technique is utilized. By offsetting the center position along these vectors in view space, the resulting quad always remains orthogonal to the camera's look-at vector.

```
void main()
{
    // Create circular particle with soft edge
    vec2 center = vec2(0.5f, 0.5f);
    float dist = distance(TexCoord, center);

    if(dist > 0.5f) discard;

    float alpha = 1.0f - (dist / 0.5f);
    alpha = alpha * alpha * alpha;

    vec3 glowColor = ParticleColor.rgb * 1.5f;

    FragColor = vec4(glowColor, ParticleColor.a * alpha);
}
```

The **Fragment Shader** (FS) processes the generated quads to create a soft, glowing spherical appearance. Instead of using external textures, a circular mask is generated procedurally. The shader calculates the Euclidean distance (dist) from the fragment's texture coordinates to the center (0.5, 0.5). Fragments where dist > 0.5 are discarded. To avoid harsh edges and simulate a "glow" profile, a non-linear decay function is applied to the alpha channel。

Rocket tail flame via Geometry Shader and fragment shader:

```
glm::vec3 localBottomPos = glm::vec3(0.0f, -1.5f, 0.0f);
glm::mat4 rotationMatrix = glm::rotate(glm::mat4(1.0f), glm::radians(rotatez - 90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
glm::vec3 rotatedBottomPos = glm::vec3(rotationMatrix * glm::vec4(localBottomPos, 1.0f));
glm::vec3 rocketBottomPos = rocket_pos + rotatedBottomPos;
```

Local Offset: An emission point is defined at localBottomPos = (0.0, -1.5, 0.0), representing the position of the bottom of the rocket's model space.
Rotational Alignment: A rotation matrix is constructed using the rocket's current orientation (rotatez). This matrix is applied to the local offset to align the emitter with the rocket's tilt.
World Positioning: The final emission coordinate (rocketBottomPos) is calculated by adding the rotated offset to the current rocket_pos in world space.

```
int emitCount = 8;  // More particles per frame for denser flame
for (int i = 0; i < emitCount; i++) {
    int idx = emissionIndex % MAX_PARTICLES;

    float angle = (rand() % 360) * 3.14159f / 180.0f;
    float speed = 30.0f + (rand() % 100) / 5.0f;  // Much faster particles

    particles[idx].position = rocketBottomPos;

    glm::vec3 baseVel = glm::vec3(
        cos(angle) * speed * 1.0f,
        0.0f,
        sin(angle) * speed * 1.0f
    );
    particles[idx].velocity = baseVel + flameDir * speed * 1.5f;
    particles[idx].color = glm::vec4(1.0f, 0.5f, 0.0f, 1.0f);  // Orange
    particles[idx].age = 0.0f;
    particles[idx].maxAge = 1.0f;  // Shorter life for tighter trail

    emissionIndex++;
}
if (particleCount < MAX_PARTICLES) particleCount = MAX_PARTICLES;
```

To prevent the flame from appearing as a single thin line, a radial spread is added. By using cos(angle) and sin(angle) in the X and Z axes, particles are dispersed within a circular cross-section.
To maintain a dense and focused plume rather than a scattered cloud, we applied strict temporal constraints. High Emission Rate: The system emits 8 particles per frame (emitCount = 8) Short Lifespan: The maxAge is set to a short duration (1.0s). This prevents the tail from becoming too long. Circular Buffer Management: Particles are stored in a fixed-size array. The emission Index uses a modulo operation to overwrite the oldest particles, creating a continuous loop of birth and death without increasing memory overhead.

And finally we'll do the same thing to **pass these particles to Geometric shader and fragment shader** to transform point primitives into camera-facing quad, assigning color and show on the screen.

Discussion

Mega_knight walk

- Proposer (蔡承志): Use blender to make skeleton animation, export as .fbx file, using HW3 animation version to read the object.

- Critic (李佑軒): Spend too much time and need to change object class (from static "Object" to animation "AnimatedModel")

- Negotiator (蔡凱旭): Still stay in static "Object" class, use vertex shader to simulate the walking animation.

Uncolour model

- Proposer (李佑軒): Need crown and princess to make scene more exquisite, make story complete.

- Critic (蔡凱旭): Only find uncolour model (.stl), maybe should use other object to replace them.

- Negotiator (蔡承志): Use blender to color the object on our own.

Coordinate finding

- Proposer (蔡凱旭): We need to find the coordinate of castle to put the object on correct position

- Critic (蔡承志): Need lot of time if we directly Trial and Error

- Negotiator (李佑軒): Write a function to output the position of camera, by moving the camera to the desired position, we can get the correct coordinate.

Uniqueness / Creativity

We implement this project based on HW3. We modify the code and add more shaders and object elements to it.
Rocket Tail frame: A high-velocity, conical plume that dynamically reacts to the rocket's orientation and speed.
Thermal Explosion: A multi-stage color-ramp simulation that transitions from high-energy particle to dissipating smoke, providing a satisfying sense of impact.
Emotes
Sending emotes is a crucial part in the game.

To bridge the gap between 2D digital media and 3D environments, we implemented a custom system for **dynamic GIF playback within the 3D scene**.This involved managing frame-by-frame texture updates in real-time and display it in the 3D world, allowing us to display animated environmental elements that add life to the static world

Idle behavior:
In most games, characters have their own idle behaviors, and we bring this into our animation to give it more “liveness” or make it closer to real gameplay by make Usagi and Chiikawa stretch and bounce slowly all the time.
Movement Cycles: We make the mega knight swing his legs while moving to make his walking looks more realistic.
The Victory Crown: Upon a successful explosion, a golden crown is procedurally generated at the site of impact, serving as a high-reward visual feedback for victory. A core creative theme of our project is that **"The BM (Bad Manners) player shall eventually be punished."**

Work Assignment

李佑軒: geometry shader for particle explode, story thinking
蔡凱旭: load static model and append vertex shader function
蔡承志: blender using and adjust object

Results

Github: https://github.com/Yu-Hsuan-1220/NYCU_ICG_Final_Project
Youtube: https://www.youtube.com/watch?v=rK46j1RQTGY

References

Model

Princess: https://sketchfab.com/3d-models/princess-b9eb9fab96fa40d587bf8e678607da20

Crown: https://www.myminifactory.com/object/3d-print-clash-royale-crown-76718

Mega_knight: https://sketchfab.com/3d-models/mega-knight-clash-royale-reflective-model-f9433b282528498a94c8a48f3fe9210a

Royal Arena (Clash Royale): https://sketchfab.com/3d-models/arena-7-royal-arena-clash-royale-c02abc1d56914736b4b042c1993e3760

Rocket: https://www.turbosquid.com/3d-models/3d-rocket---clash-royale-model-1525232

Usagi:https://sketchfab.com/3d-models/usagi-edeb7465e9894aa7b0d205216ffde45d

Chiikawa: https://sketchfab.com/3d-models/chiikawa-7322ae143a3e425884c238923ebbe5de